Access Control Decision Guide

Use this guide to define who can access which data and how organizational boundaries are enforced in Medplum. Work through tenancy, patient and practitioner sharing, role permissions, and administration to produce a membership and access-policy model. Use the Authentication Decision Guide for login and identity-provider choices.

Companion to the Authorization and Access Control docs.

Identify your users and organizational structure, then distinguish API restrictions from UI filtering. Work through these decisions for the access the API must enforce:

1. **How will you separate and share data?** For applications with internal or multi-organization isolation.
2. **Which data should each role be able to access?** Define each role's scope.
3. **What should each role be allowed to do?** Choose permissions and any restrictions.
4. **Who will administer users and configuration?** Assign administrative responsibilities.

Also review patient and caregiver access, third-party app access, and IP restrictions where applicable.

Before you start

Who needs access?

- Patients / caregivers only
- Providers / staff / partners (clients or outside vendors) only
- Both

How is your organization structured?

- Single organization
- Multiple locations of one organization
- Multiple independent organizations (MSO-style)
- Departments or service lines within one organization

- Combination

Does this restriction need to prevent API access, or only change what the UI displays?

Option A: Prevent access to the data

The user must not retrieve or modify the data, including through direct API requests. Enforce the restriction through AccessPolicy and ProjectMembership configuration.

Option B: Change the displayed view

The user is authorized to access the data, but the application should show a narrower view. Use UI filters or display settings within the user's permitted access.

For example, a clinician authorized for three clinics can view one clinic at a time using a UI selector. If each session must only access one clinic's data, its authenticated permissions must also be limited to that clinic.

Both can apply: the API defines the permitted access; the UI selects what to display within it. Apply this distinction to tenant boundaries, resource permissions, and individual fields throughout the guide.

Decision 1: How will you separate and share data?

For applications that need isolation between organizations, locations, teams, or service lines. If you have a single organization with no internal isolation, skip to Decision 2.

Within a project, tenant isolation uses **compartments** matched by parameterized access policies. Use $set-accounts to assign tenant references; Medplum computes meta.compartment. These tenant assignments extend the standard FHIR patient-compartment model.

Recommended pattern: Assign patients to tenants through enrollment workflows, then grant practitioners access through parameterized ProjectMembership.access entries. See Multi-Tenant Access Control.

What does a tenant represent?

Choose what a tenant represents in your application. What do you call these groups: clinics, programs, sites, or accounts?

Option A: Organizations, clinics, or practices

Use Organization when tenants represent clinics, practices, or healthcare organizations in an MSO model.

Option B: Service lines or departments

Use HealthcareService to represent functional groups within an organization.

Option C: A dedicated team for each patient

Use a CareTeam per patient.

If tenants form a hierarchy: how will access extend to child tenants?

One approach for nested tenants, such as region → clinic, is to grant access through the lowest-level tenants:

- Patients are tagged only at the **lowest level** they belong to.
- Practitioners enrolled at an intermediary level get one ProjectMembership.access entry per lowest-level tenant under it (iteratively expanded).
- Adding a new lowest-level tenant requires a workflow to update every practitioner membership at or above that intermediary.

Can a patient belong to one tenant or several?

Patient access	Approach
One tenant per patient	One tenant assignment on the Patient
Patient shared across tenants	Multiple tenant assignments on the Patient

Are practitioners visible across tenants or only within their own?

Practitioner visibility	Approach
Shared across tenants (global directory)	Allow Practitioner reads without a tenant filter
Visible only within their tenant	Tag with tenant compartment; policy filters by _compartment

Which resources are shared across tenants?

For terminology (CodeSystem, ValueSet), Questionnaires, PlanDefinitions, and ActivityDefinitions, decide whether resources are shared or tenant-specific. Also identify any resources shared across projects.

Situation	Approach
All resources of a type are shared	Allow reads without a tenant filter for that type
Tenant-specific Questionnaires	Use $set-accounts to assign the tenant; filter by _compartment

Situation	Approach
Some resources of a type are shared	Use separate criteria for authorized tenant resources and explicitly designated shared resources. An unfiltered rule would also expose other tenants' resources
Bots used across tenants	Configure access to the Bot separately from its execution permissions; by default it uses its own ProjectMembership policy
Reference data shared across projects	A super admin configures a linked project and its exported resource types; access through the link is read-only and still subject to policy

Allow the terminology resources your UI needs; overly restrictive policies can leave dropdowns empty. For Bots, see Run as User; for shared reference data, see Linked Projects.

How will patients be enrolled or transferred between tenants?

Questions:

- When does a patient get enrolled in a tenant? (registration, first appointment, referral, insurance, geography)
- Who is authorized to move a patient between tenants?
- On transfer: do records follow the patient, stay with the original tenant, or become shared?

Situation	Approach
Patient enrollment	An authorized admin or Bot calls $set-accounts with the full tenant list; use propagate: true to update existing patient-compartment resources
Transfer all patient-compartment data	Replace the old tenant in the list and propagate the change. Verify completion and any other grants before treating access as revoked
Share data with both tenants	Include both tenants in the list and propagate the change
Keep historical records with the original tenant	Design a separate record-level assignment workflow; patient-wide propagation does not distinguish historical from future records

$set-accounts requires project-admin or super-admin authority and checks access to the affected resources. Propagation follows each resource's own patient-compartment references, not arbitrary links between resources.

Reference implementation: MSO demo enrollment.ts.

Next: Define each role's data scope in Decision 2.

Decision 2: Which data should each role be able to access?

List your roles, such as physician, RN, MA, front desk, billing, care coordinator, and medical director. **If you have multiple tenants, choose the data scope for each role below.** Otherwise, continue to Decision 3.

Option A: Access to one tenant

Use a single ProjectMembership.access entry parameterized with that tenant.

Option B: Combined access to several tenants

Use one access entry per tenant with the same policy.

Option C: Organization-wide or cross-tenant oversight

Use an unparameterized policy with no _compartment filter.

Option D: Separate authenticated sessions for each tenant

Give each session a membership limited to one tenant. Switching context means switching the authenticated login, rather than changing a UI filter. Separate credentials are not always necessary: the Invite API supports creating additional memberships with forceNewMembership.

How do these policies combine?

Policy grants are additive, not a way to subtract access. Do not add a restrictive policy to narrow an existing broad grant. For overlapping resource rules, the first matching rule determines field restrictions and write constraints; design and test those rules together.

Implementation: Parameterized Policies, Invite API.

Next: Choose what each role can do with that data in Decision 3.

Decision 3: What should each role be allowed to do?

Which resource types and interactions does the role need?

For each role, list the resource types it needs and choose which interactions it can perform, such as reading, creating, updating, searching, or deleting.

Permission option	Approach
All supported FHIR interactions	Standard resource entry, no interaction restriction
Read-only	readonly: true
Selected interactions, such as create-only or read without search	Explicit interaction array

Does the role need additional restrictions?

Apply only the restrictions your roles need.

Should any fields be hidden or read-only?

Use field-level restrictions for specific elements, such as a Patient's address.

Situation	Approach
Readable but not writable	readonlyFields
Hidden from returned resources	hiddenFields; these restrictions also affect writes, not just returned output
No access to a clinical resource type	Omit that type from the role's grants; ensure no broader rule grants access

Field masking is not a substitute for denying access to a resource. For presentation-only changes, hide fields in the UI.

Should writes enforce required fields or state transitions?

Use write constraints to enforce these rules at the policy layer:

Situation	Approach
Prevent a field changing after a terminal state	Use writeConstraint to check the prior state and compare the field before and after
Require a field when entering a state	Check %after and field presence in a writeConstraint
Apply a constraint only to updates	Prefix with %before.exists() implies ...
Apply a constraint only to creates	Prefix with %before.exists().not() implies ...

These prefixes control when a constraint applies; use interaction to allow only creates or updates. Keep FHIRPath simple. If a Bot enforces complex rules, use a controlled invocation path; a Subscription-triggered Bot runs after the write and cannot reject it.

Should access depend on resource attributes?

Use criteria-based filtering for attributes such as geography, code, or status:

Situation	Approach
Patient address filtering	Patient?address-state=%state, parameterized
Several allowed states	One access entry per state
Specialty or status filter	criteria with code/status filter
Criteria + tenant	Single criteria string combining both

Address filtering restricts records; it does not validate practitioner licensure. criteria supports a subset of FHIR search, with :not and :missing modifiers and no chained searches. Denormalize or assign tenant references through an enrollment workflow where needed.

Files: Binary resources do not support policy criteria; set an appropriate securityContext. See Binary Security Context.

Implementation: Access Policies, Licensure and Scheduling.

Should the UI change based on the role?

If your frontend shows or hides UI based on the user's role.

Situation	Approach
UI changes based on role	Read the effective policy's basedOn references from /auth/me

basedOn identifies the policies used to build the effective policy; it is not a policy-inheritance setting. Use it to inform the UI, while continuing to enforce access at the API.

Next: Assign administrative responsibilities in Decision 4.

Decision 4: Who will administer users and configuration?

Decide who manages users and memberships, who manages Bots, ClientApplications, and AccessPolicies, and whether those administrators should see clinical data. Identify any server-level needs, such as project creation or overwriting protected fields.

Situation	Approach
Trusted project administrator	ProjectMembership.admin grants project administration capabilities
Admin with limited routine clinical access	Assign an explicit custom AccessPolicy limiting clinical resources

Situation	Approach
A role that must not access clinical content	Use a non-admin role with narrowly scoped permissions; do not rely on hiddenFields or an admin's clinical policy as a hard boundary
Project creation or protected server-level changes	Super-admin operations; for hosted Medplum, coordinate with Medplum
Separate user-management and technical duties	Define the permitted operations and use controlled backend workflows where needed; stacking policies does not subdivide the admin flag

Project admins are trusted administrators. They can manage access policies and memberships, and act on behalf of users. A clinical policy limits their routine access but does not prevent them from changing grants. The admin flag does not bypass clinical policies; omitting a policy can still grant broad access.

Default or custom policy? The built-in Admin default grants broad read/write access. When admin status changes, Medplum swaps the recognized Practitioner/Admin default to the matching default, if configured. Custom policy assignments are unchanged by this swap; the admin flag still changes administrative capabilities. Use an explicit custom policy when the defaults are too broad.

Implementation: Admin Users, Default Access Policies, On-Behalf-Of.

Next: Review the additional decisions below that apply to your application.

Additional decisions, if applicable

How will patients and caregivers get access?

For applications with a patient portal.

Choose how patients register, whether caregivers or proxies need access (such as parents, adult children, or guardians), and how those relationships are established and revoked.

How will patients register?

Registration option	Approach
Invite-only	Use the Invite API with a patient-scoped policy
Open self-registration	Set Project.defaultPatientAccessPolicy; this enables open registration

Check both enrollment paths. Open registration uses Project.defaultPatientAccessPolicy. Patient invitations without an explicit policy prefer the Patient entry in Project.defaultAccessPolicies, falling back to defaultPatientAccessPolicy if that entry is absent.

Align these defaults if both paths should grant the same access. Changing the registration default affects future registrations; it does not migrate existing memberships. See Default Access Policies.

Which patient records can each person access?

Access option	Approach
Patient sees own data	Parameterized policy with %patient on clinical resources
Single caregiver, single patient	One access entry with %patient
One caregiver, multiple patients	Multiple access entries, different %patient
Multiple caregivers per patient	Each caregiver has their own ProjectMembership
Revoke caregiver	Remove the corresponding grants and verify API access is revoked
Time-bound caregiver access (e.g., expires at age 18)	Automate removal of the corresponding membership grants at expiry; hiding access in the UI is insufficient

Implementation: Patient and Caregiver Policies, Open Patient Registration.

How will third-party apps get access?

If an app needs SMART launch or delegated access. SMART is commonly used for third-party integrations, but is not limited to them.

Choose patient-context, provider-context, or standalone launch. Define the scopes the app needs and whether it needs offline access.

Situation	Approach
Your own frontend without SMART launch requirements	Use a standard ClientApplication and AccessPolicy
App launched from an EHR or portal	Request launch plus the clinical data scopes the app needs
Standalone app needing patient selection	Request launch/patient plus patient data scopes, such as patient/*.rs
Patient data access	Use patient/ scopes with patient context; narrow the resource types where possible

Situation	Approach
Access within the current user's permissions	Use user/ scopes and the user's AccessPolicy
Access after the user is offline	Request offline_access for a refresh token

Launch scopes request context; they do not grant clinical data access. Clinical scopes work alongside AccessPolicy restrictions. Use .rs for read/search access or .cruds for full access within the policy; enforce finer interaction restrictions through AccessPolicy.resource.interaction rather than assuming arbitrary SMART permission-letter combinations are enforced.

Implementation: SMART Scopes.

Should access be restricted to specific networks?

If access should be restricted by IP range.

Choose which networks can connect, such as a VPN or clinic network, and whether different roles need different rules.

Situation	Approach
VPN-only	Allow VPN ranges + wildcard block
On-prem only	Allow clinic network + wildcard block
Mixed	Different AccessPolicies per role

Use the documented IPv4 address or address-prefix rules, not CIDR notation. The first matching rule wins; end an allowlist with a wildcard block.

These rules apply to supported authentication paths, but are not enforced on refresh-token requests or rechecked on every bearer-token API call. Do not use them as the sole control for a requirement that every request originate from an approved network.

Implementation: IP Access Rules.

Decisions to Record

- **Tenant boundaries:** Isolated and shared data, including handling of patient transfers.
- **Role permissions:** Allowed data and actions, distinguishing API restrictions from UI filters.
- **Automation identity:** Identity and permissions for each Bot or backend service, where applicable.
- **Access lifecycle:** Owners for grants, changes, revocation, expiry, and transfers.
- **API acceptance checks:** Requests that should succeed or fail for each role, including after revocation.